EXPERIMENT-13 Minimax Algorithm for Gaming

Aim

To implement the **Minimax algorithm** in Python for selecting the optimal move in a two-player game.

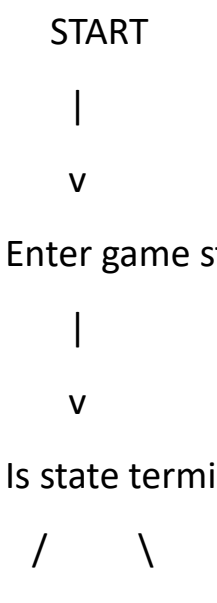
Objectives

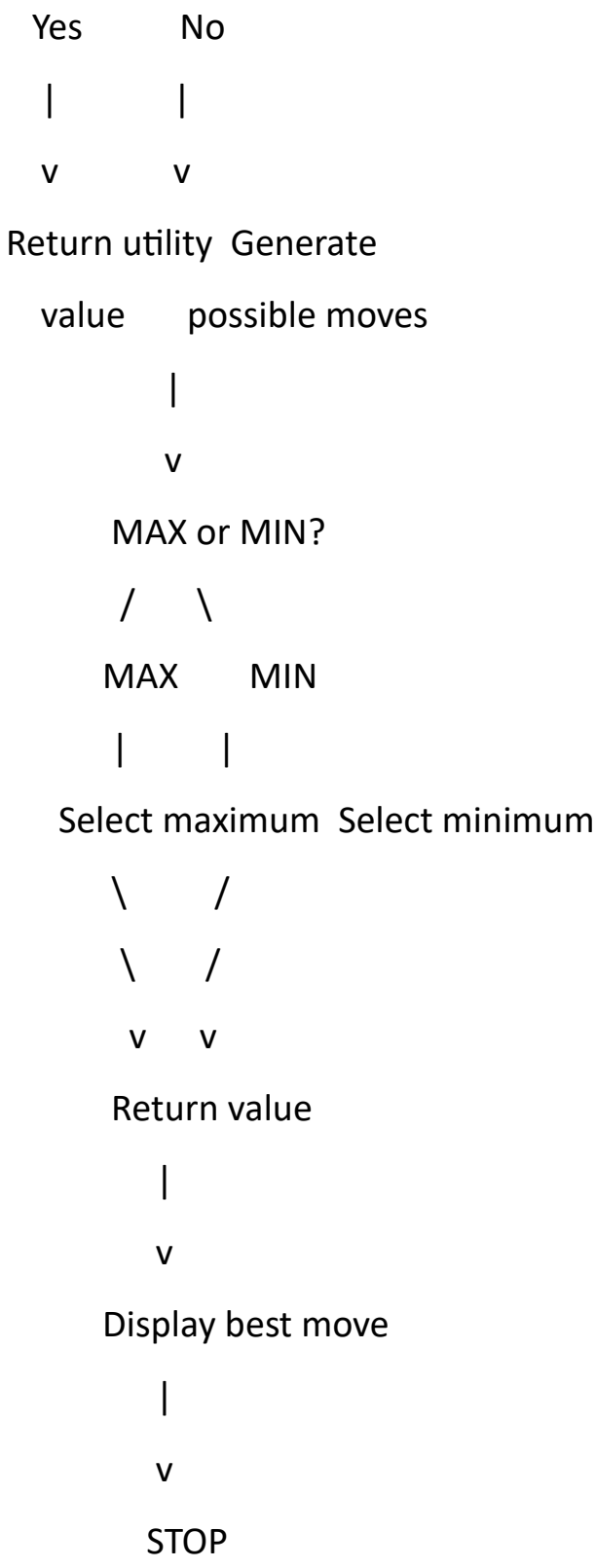
- To understand adversarial search.
- To implement MAX and MIN decision-making.
- To select the best possible move assuming the opponent also plays optimally.

Algorithm

1. Start with the current game state.
2. Generate all possible moves.
3. If the state is terminal, return its utility value.
4. For MAX player, select the maximum value.
5. For MIN player, select the minimum value.
6. Continue recursively until the best move is found.
7. Display the optimal move.

Flowchart




Yes
No
Return utility value
Generate possible moves
MAX or MIN?
MAX
MIN
Select maximum
Select minimum
Return value
Display best move
STOP

Python Program

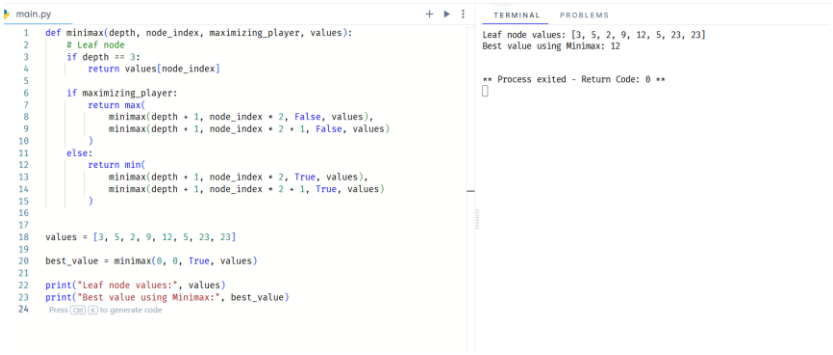
```
def minimax(depth, node_index, maximizing_player, values):
    # Leaf node
    if depth == 3:
        return values[node_index]

    if maximizing_player:
        return max(
            minimax(depth + 1, node_index * 2, False, values),
            minimax(depth + 1, node_index * 2 + 1, False, values)
        )
    else:
        return min(
            minimax(depth + 1, node_index * 2, True, values),
            minimax(depth + 1, node_index * 2 + 1, True, values)
        )



values = [3, 5, 2, 9, 12, 5, 23, 23]

best_value = minimax(0, 0, True, values)

print("Leaf node values:", values)
print("Best value using Minimax:", best_value)
```

Output

Leaf node values: [3, 5, 2, 9, 12, 5, 23, 23]

Best value using Minimax: 12

Output:



Result

The Minimax algorithm successfully determines the **optimal value as 12**.

Conclusion

Thus, the **Minimax algorithm** was successfully implemented to determine the best decision for a player assuming an optimal opponent.